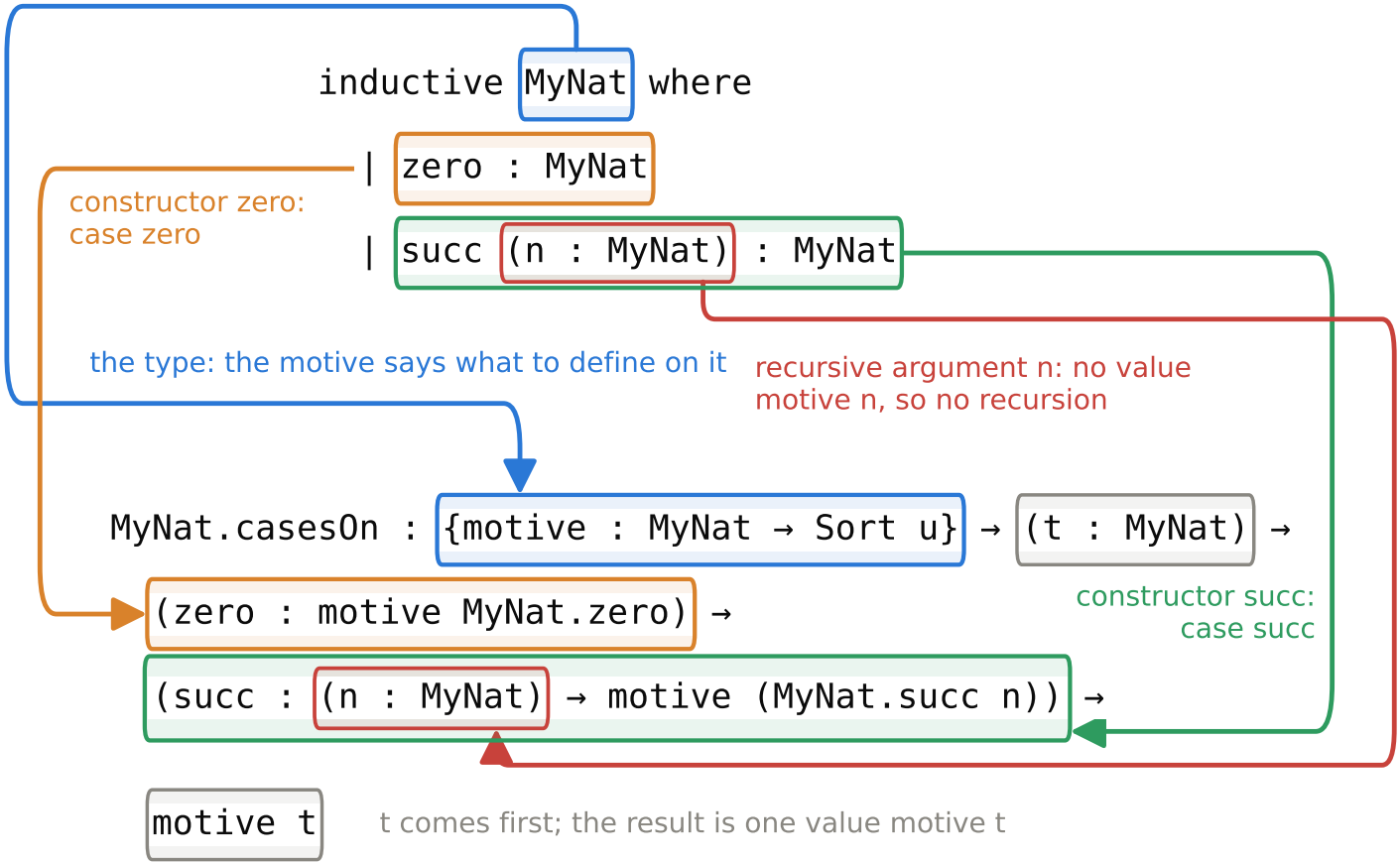
inductive MyNat where
| zero : MyNat
| succ (n : MyNat) : MyNat
constructor zero:
case zero
the type: the motive says what to define on it
recursive argument n: no value
motive n, so no recursion
MyNat.casesOn : {motive : MyNat → Sort u} → (t : MyNat) →
(zero : motive MyNat.zero) →
constructor succ:
case succ
(succ : (n : MyNat) → motive (MyNat.succ n)) →
motive t
t comes first; the result is one value motive t